Understanding the ASP.NET Core Web API and SQL Server Architecture

1 Comment / By Dot Net Tutorials / November 23, 2024

Back to: Microsoft Azure Tutorials

Understanding the ASP.NET Core Web API and SQL Server Architecture

In Chapter 1, we understood what deployment means, why cloud hosting is important, and why Microsoft Azure is widely used for modern application hosting. Now, before we move further toward Azure deployment, Docker, and Kubernetes, we must clearly understand the application's internal architecture.

When we say **ASP.NET Core Web API + SQL Server Architecture**, we mean the complete internal flow of the application: who sends the request, where the request enters, which class handles the business rules, how data is read or saved, how SQL Server stores the data, how configuration is managed, and how the application and database communicate securely.

What Do We Mean by Architecture in ASP.NET Core Web API?

When we say architecture, we mean the overall structure of the application and how different parts of the system interact. In simple words, architecture is the blueprint of the application. Just like a building has:

- Foundation
- Rooms
- Electrical wiring
- Water pipelines
- Doors and windows

Similarly, an ASP.NET Core Web API project also has:

- Controllers
- Business logic
- Data access code
- Models
- DTOs
- Database
- Configuration

If these parts are not organized properly, the project may work for a small demo, but it becomes difficult to maintain, test, secure, and deploy in real-world scenarios. So, architecture is not just about code. It is about writing code in a structured and manageable way.

Real-Time Understanding of the Complete Flow

Let us first understand the entire flow of an ASP.NET Core Web API with SQL Server in the simplest possible way. Suppose we are building a Product Management System API. A frontend application or client sends a request such as: **GET /api/products**

Now the flow usually happens like this:

1. The request comes to the API Endpoint.
2. The controller receives the request.
3. The controller calls the service layer.
4. The service layer applies business rules if needed.
5. The service layer calls the repository.
6. The repository communicates with SQL Server through Entity Framework Core.
7. SQL Server returns the requested data.
8. The data goes back through the repository and service.
9. The controller sends the final response to the client in JSON format.

This layered flow is one of the most common patterns used in modern ASP.NET Core Web API applications.

How ASP.NET Core Web API Works?

ASP.NET Core Web API is a framework for building HTTP-based services. These services expose endpoints that can be consumed by different kinds of clients. A Web API does not usually return HTML pages like a normal website. Instead, it returns data, commonly in JSON format.

For example:

- A mobile app may call the API to get product details
- An Angular or React frontend may call the API to display a list of users
- Another microservice may call the API to fetch order information
- Swagger may call the API for testing during development

Simple Example:

Suppose you build a **Product Management API**. It may expose endpoints such as:

- Get all Products
- Get Product by ID
- Add a Product
- Update a Product
- Delete a Product

When a client calls one of these endpoints, the API processes the request, talks to SQL Server if needed, and returns the result. So, an ASP.NET Core Web API acts like a middle layer between the client and the database.

Main Building Blocks of the Architecture

Now, let us understand the key components of an ASP.NET Core Web API architecture.

Controllers

Controllers are the entry points of the API. They receive HTTP requests from clients and return responses. In simple terms, the controller is the first internal component to handle an incoming API call.

Its responsibilities usually include:

- Receiving the incoming HTTP request
- Reading route values, query string values, or request body data
- Calling the appropriate service method
- Returning the final HTTP response

A controller should remain thin. It should not contain heavy business logic or too much database code.

Responsibility of a Controller

If a client sends a request to create a new product, the controller should:

- Accept the request data
- Validate basic input if needed
- Call the service layer
- Return a success or failure response

It should not contain complicated pricing logic, discount rules, stock logic, or SQL queries directly.

Services

Services contain the business logic of the application. This is one of the most important layers in a real-time project. A service decides:

- What should happen?
- What validations are needed?
- What business rules must be applied?
- What response should be prepared?

Example

Suppose we are creating a new employee. The service may check:

- Is the employee's email already used?
- Is the salary valid?
- Is the department active?
- Should a welcome email be sent?
- Should audit information be stored?

These are business rules, and they belong in the service layer, not inside the controller.

Why Services Are Important?

The service layer keeps the application logic organized and reusable. If we place all business rules inside controllers, the code becomes difficult to manage. But when business rules are placed inside services, the project becomes cleaner and easier to maintain. Services make the project:

- Cleaner
- Easier to maintain
- Easier to test
- Easier to extend later

Repositories

Repositories are responsible for data access. Their main job is to communicate with the database and perform CRUD operations such as:

- Insert
- Update
- Delete
- Retrieve

In many applications, repositories work with Entity Framework Core and DbContext.

Example:

A repository may:

- Fetch all products
- Find a product by ID
- Add a new product
- Delete an existing product
- Apply filters
- Save changes

Why is the Repository Layer used?

The repository layer helps separate database logic from business logic.

That means:

- Services focus on business rules
- Repositories focus on data access

This makes the code more organized.

Database

The **Database** is where application data is stored permanently.

Examples of stored data:

- Users
- Products
- Categories
- Orders
- Students
- Employees
- Payments
- Bookings

In our course context, the database is **SQL Server**. The API does not permanently store data inside itself. It depends on the database for storing and retrieving data.

What Is the Role of SQL Server?

SQL Server is the relational database management system used to store and manage application data.

Its role includes:

- Storing tables
- Managing relationships between tables
- Enforcing constraints
- Processing queries
- Supporting inserts, updates, and deletes
- Ensuring data integrity
- Supporting indexing, transactions, and security features

Example

Suppose we have a Product API. SQL Server may store:

- Products table

- Categories table
- Brands table
- Orders table

Then:

- When the API wants all products, it queries SQL Server.
- When the API wants to save a new product, it inserts it into SQL Server.
- When the API updates stock, SQL Server stores the latest value.

So SQL Server is the application's persistent data store.

What Are API Endpoints?

An **API endpoint** is a specific URL through which a client can perform a specific operation of the API. In simple words, API endpoints are the URLs through which clients communicate with the API. Endpoints define:

- What operation can be performed
- What input is required
- What response will be returned

For example:

- GET /api/products
- GET /api/products/10
- POST /api/products
- PUT /api/products/10
- DELETE /api/products/10

Why Endpoints Matter

Endpoints are the public interface of the API. This is how clients talk to the backend.

- A client does not know how your service layer works internally.
- A client does not know what repository you are using.
- A client does not know the internal structure of the database
- A client only knows the endpoint and the contract.

That is why endpoint design should be clean, meaningful, and consistent.

What Are Clients?

A **Client** is any application or tool that consumes the API. A client can be:

- A web frontend
- A mobile application
- Another API
- A desktop application
- Swagger UI
- Postman
- Fiddler

Example

Suppose your ASP.NET Core Web API is used for a school management system.

Then the clients may be:

- Admin panel frontend
- Teacher mobile app
- Student portal
- Reporting application

All these clients can call the same API endpoints. So, API clients are not part of the API project itself. They are separate systems that send requests to the API. In simple words, the API is the provider, and the client is the consumer.

What Is a Connection String?

A **Connection String** is a configuration setting that tells the application how to connect to the database. It usually contains information such as:

- Server name
- Database name
- Authentication details
- Trust and certificate options depending on the environment

In simple terms, a connection string is the address and access information needed to connect to the database. Without it, the API does not know:

- Where the database is
- Which database to use
- How to log in

Example

When the API starts and tries to use DbContext, it needs to know:

- Is SQL Server on the local machine?
- Is it on another server?
- Is it Azure SQL Database?
- Which database should it connect to?
- What credentials should it use?

All of that is defined through the connection string.

Important Note

Connection strings are extremely sensitive. They should never be hardcoded directly inside controllers or service classes. They should be stored in:

- appsettings.json
- Environment variables
- Azure App Settings
- Azure Key Vault in advanced scenarios

What is appsettings.json?

appsettings.json is the main configuration file for ASP.NET Core applications. It is commonly used to store:

- Connection strings
- Logging settings
- Custom application settings
- External service URLs

Why It Is Useful

Instead of writing configuration values directly inside code, ASP.NET Core encourages us to keep them in configuration files. This makes the project:

- Cleaner
- Easier to change
- Easier to manage across environments

Example Use Cases

You can store the following values in appsettings.json:

- Database Connection String
- JWT settings
- Email settings
- Third-Party API keys
- Application-Specific Configuration

- Logging Configuration

What Do We Mean by Environment-Based Configuration?

Environment-based configuration means the application can use different settings depending on the environment in which it is running. In real-world applications, we do not use the same configuration for every environment.

For example:

- The development environment may use a local SQL Server
- The testing environment may use a Staging Database
- Production environment may use Azure SQL Database

That is why ASP.NET Core supports environment-based configuration.

Common Environment Files

You may see files such as:

- appsettings.json
- appsettings.Development.json
- appsettings.Production.json

Why This Is Useful

This allows the application to behave differently in different environments without changing the main code.

For example:

- A development environment may enable detailed errors
- The production environment may hide sensitive error details
- The development environment may use test data
- Production may use live database settings
- A development environment may enable the Swagger endpoint
- Production environment should disable the Swagger endpoint

This separation is very important because the settings used on a developer machine should not always be used in production.

Difference Between Hosting the Application and Hosting the Database

This is another very important concept. Many beginners think the application and database are always in the same place. That is not necessary.

Hosting the Application

Hosting the application means deploying the ASP.NET Core Web API somewhere, such as:

- local IIS
- Azure App Service
- Docker container
- AKS

This is where the API code runs. So, hosting the application means deploying the ASP.NET Core Web API so that clients can call its endpoints.

Hosting the Database

Hosting the database means deploying **SQL Server or Azure SQL Database** somewhere, such as:

- local SQL Server
- On-premises SQL Server
- SQL Server on a virtual machine
- Azure SQL Database

This is where the data is stored. So, hosting the database means running SQL Server or Azure SQL to store and manage data properly.

Summary

In this chapter, we examined the internal architecture of an ASP.NET Core Web API application that works with SQL Server. We learned that the Web API acts as the middle layer between clients and the database, and we clearly understood the role of controllers, services, repositories, DbContext, SQL Server, endpoints, clients, connection strings, and configuration files.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Consent

Do Not Sell My Information